

Informe: Taller Monitoreo

Monitoreo y réplica automática de un servidor FTP

Gabriel Jaramillo Cuberos Roberth Méndez Jorge Olaya
Luz Adriana Salazar Guden Sebastián Silva Santiago Galindo

Agosto 2026

Repositorio: https://github.com/GabrielJaramilloCuberos/Arquitectura-de-Software/tree/main/Taller_1

Índice

1. Análisis del problema	2
2. Arquitectura de la solución	2
3. Decisiones arquitectónicas	2
4. Implementación del monitoreo — Parte A (Apache Commons Net)	3
5. Implementación del monitoreo — Parte B (Apache Camel)	4
6. Manejo de errores	5
7. Comparación Parte A vs. Parte B	5
8. Dificultades y soluciones	6
9. Conclusiones	7
10. Pruebas	8

1. Análisis del problema

El ejercicio plantea un escenario típico de **integración de sistemas**: existe un servidor FTP donde distintos procesos depositan archivos, y se necesita **monitorear ese servidor de forma continua** para detectar archivos nuevos y **replicarlos automáticamente** en una carpeta local, sin intervención manual.

A partir de este enunciado se identificaron los siguientes requisitos:

- **Monitoreo continuo (polling)**: el sistema debe revisar el servidor a intervalos regulares y configurables, no una sola vez.
- **Recorrido recursivo**: los archivos pueden estar organizados en subdirectorios dentro del FTP; el monitoreo debe alcanzarlos a todos.
- **No duplicar descargas**: un archivo ya descargado no debe volver a copiarse en el siguiente ciclo de polling.
- **Configuración externa**: host, credenciales, directorio remoto, destino local e intervalo de polling deben poder cambiarse sin recompilar el código.
- **Entorno reproducible**: se necesita un servidor FTP de pruebas que cualquier integrante del equipo pueda levantar igual, sin depender de un FTP externo.
- **Comparar dos enfoques de implementación**: uno de bajo nivel (control manual del protocolo) y uno de alto nivel (framework de integración), para evaluar trade-offs de arquitectura.

Estas dos últimas necesidades dieron origen a la separación del ejercicio en **Parte A** (implementación manual con Apache Commons Net) y **Parte B** (implementación declarativa con Apache Camel), ambas resolviendo el mismo problema desde paradigmas distintos.

2. Arquitectura de la solución

La Figura 1 resume cómo se relacionan los componentes de la solución: un servidor FTP contenerizado, un archivo de configuración compartido, y dos aplicaciones independientes (Parte A y Parte B) que hacen polling sobre el mismo servidor y escriben en carpetas de destino separadas.

Ambas implementaciones comparten:

- El **mismo servidor FTP** contenerizado (`ServidorFTP/docker-compose.yml`).
- El **mismo archivo de configuración** (`taller/src/main/resources/config.properties`).
- El **mismo proyecto Maven**, pero en paquetes independientes (`com.arqui.ParteA` y `com.arqui.ParteB`), de modo que cada una se puede ejecutar de forma aislada.

3. Decisiones arquitectónicas

Decisión	Justificación
Servidor FTP contenerizado con Docker Compose (<code>pure-ftpd</code>)	Entorno de pruebas reproducible e idéntico para todo el equipo, sin instalar ni configurar un FTP real en cada máquina.

Decisión	Justificación
Configuración externalizada en <code>config.properties</code>	Permite cambiar host, credenciales, directorio remoto, carpeta destino e intervalo de polling sin tocar ni recompilar el código, en línea con el principio de separar configuración del código.
Un solo proyecto Maven con dos paquetes (ParteA , ParteB) en vez de dos proyectos separados	Reutiliza dependencias y configuración comunes y facilita comparar ambas soluciones lado a lado con el mismo <code>pom.xml</code> .
Dos paradigmas de implementación distintos para el mismo problema	Permite comparar objetivamente un enfoque imperativo/manual (control total del protocolo FTP) contra un enfoque declarativo basado en EIP (Enterprise Integration Patterns) con un framework de integración, evaluando productividad, resiliencia y mantenibilidad.
Polling en vez de notificación por eventos	El protocolo FTP no soporta notificaciones push de archivos nuevos; el polling periódico es el mecanismo estándar para este tipo de integración.
Cada parte escribe en su propia subcarpeta (ParteA/CopiasA , ParteB/CopiasB)	Evita que ambas implementaciones se pisen entre sí al correr en paralelo sobre el mismo destino local, facilitando la comparación de resultados.

4. Implementación del monitoreo — Parte A (Apache Commons Net)

Archivo: `taller/src/main/java/com/arqui/ParteA/AppA.java`

Funcionamiento:

1. Se carga `config.properties` desde el classpath.
2. Se abre **una sola conexión FTP** (`FTPClient`), en modo pasivo y binario, que se mantiene abierta durante toda la ejecución.
3. Un bucle infinito (`while (true)`) ejecuta `procesarDirectorio(...)` y luego espera `poll.interval.secs` segundos con `Thread.sleep(...)`.
4. `procesarDirectorio` lista el contenido del directorio remoto con `ftp.listFiles(...)` y:
 - si la entrada es un directorio, se llama **recursivamente** sobre él (Commons Net no ofrece un listado recursivo nativo);
 - si es un archivo, se intenta descargar.
5. La **deduplicación** se controla manualmente con un `Set<String>archivosDescargados` en memoria, indexado por la ruta remota completa del archivo: si ya está en el set, se omite.
6. La descarga usa `ftp.retrieveFile(rutaRemota, out)` volcando el contenido a un `OutputStream` local, creando los directorios destino si no existen.

Esta implementación expone y resuelve **explícitamente** cada aspecto del problema (recursión, deduplicación, ciclo de polling), lo cual da control total pero traslada toda la responsabilidad al código propio.

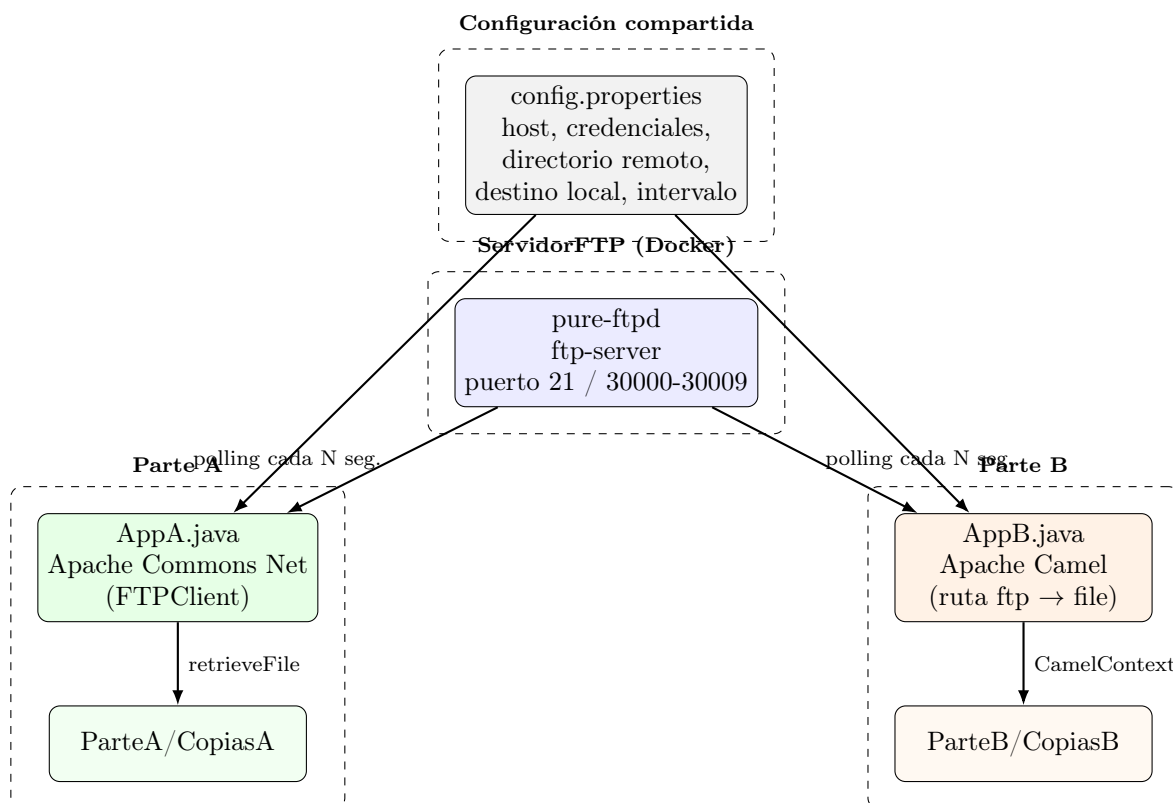


Figura 1: Arquitectura general de la solución.

5. Implementación del monitoreo — Parte B (Apache Camel)

Archivo: `taller/src/main/java/com/arqui/ParteB/AppB.java`

Funcionamiento:

1. Se carga la misma `config.properties`.
2. Se define una **ruta de Camel** (patrón EIP `from → to`) dentro de un `CamelContext`:

```
from("ftp://host:puerto/directorio?...")
.to("file:destino?autoCreate=true&fileName=${file:name}")
.process(exchange -> imprimirArchivoDescargado(exchange));
```

3. Todo el comportamiento de polling, recursión y deduplicación se delega a **parámetros del endpoint FTP** de Camel, en vez de código propio:
 - `delay=<ms>` → intervalo de polling (equivalente al `Thread.sleep` de la Parte A).
 - `recursive=true` → recorrido recursivo de subdirectorios.
 - `idempotent=true` → Camel usa un repositorio idempotente en memoria para no reprocesar archivos ya descargados (equivalente al `Set` manual de la Parte A).
 - `noop=true` → el archivo **no se borra ni se mueve** en el FTP tras descargarlo (por defecto camel-ftp elimina/renombró el archivo origen al procesarlo).
 - `binary=true` y `passiveMode=true` → equivalentes a `setFileType(FTP.BINARY_FILE_TYPE)` y `enterLocalPassiveMode()` de la Parte A.
4. El proceso principal se mantiene vivo con `new CountDownLatch(1).await()`, ya que el propio `CamelContext` corre el polling en sus propios hilos internos.

5. Un *shutdown hook* invoca `contexto.stop()` para cerrar la ruta de forma ordenada al recibir `Ctrl+C` o una señal de terminación.

Esta implementación traduce el problema a un **flujo declarativo**: en lugar de programar cómo recorrer, deduplicar y descargar, se **configura** un endpoint que ya sabe hacerlo.

6. Manejo de errores

Aspecto	Parte A (Commons Net)	Parte B (Camel)
Falta de <code>config.properties</code>	<code>RuntimeException</code> explícita al iniciar (falla rápido, <i>fail-fast</i>).	<code>RuntimeException</code> explícita al iniciar (mismo mecanismo, mismo criterio).
Error de red/IO durante un ciclo de polling	No se captura : la excepción se propaga fuera del <code>while(true)</code> , terminando el proceso completo. No hay reintentos ni reconexión automática.	El <i>consumer</i> de Camel captura los errores de cada poll internamente (<code>DefaultErrorHandler</code>); un fallo puntual se registra en el log pero no detiene la ruta , que reintenta en el siguiente ciclo.
Reconexión ante caída del servidor FTP	No implementada; requeriría envolver el ciclo en <code>try/catch</code> y volver a llamar a <code>conectarFTP</code> .	Manejada por el framework como parte del ciclo de vida del <i>consumer</i> .
Cierre ordenado del proceso	No aplica (no hay recursos que liberar de forma especial más allá de la conexión FTP).	<i>shutdown hook</i> + <code>contexto.stop()</code> para liberar hilos y conexiones de Camel de forma limpia.
Archivos con nombre/ruta inválida en Windows vs. Unix	No aplica directamente (uso de <code>Path/Paths</code> de Java).	Se normaliza manualmente el separador de rutas (<code>replace("\\", "/")</code>) antes de construir el endpoint <code>file:</code> .
Caracteres especiales en usuario/contraseña	No aplica (se pasan como parámetros del método <code>login</code> , no como parte de una URI).	Se codifican con <code>URLEncoder.encode(...)</code> , porque usuario y contraseña forman parte de la URI del endpoint FTP y podrían romperla.

Conclusión de esta sección: la Parte A tiene un manejo de errores **mínimo y deliberadamente simple** (fail-fast también ante errores de red, deteniendo todo el proceso), mientras que la Parte B hereda un manejo de errores **más robusto “gratis”** por apoyarse en un framework maduro, a cambio de menor visibilidad directa sobre qué ocurre internamente ante un fallo.

7. Comparación Parte A vs. Parte B

Dimensión	Parte A — Apache Commons Net	Parte B — Apache Camel
Nivel de abstracción	Bajo (API imperativa sobre el protocolo FTP)	Alto (patrón EIP declarativo <code>from → to</code>)

Dimensión	Parte A — Apache Commons Net	Parte B — Apache Camel
Recorrido recursivo	Implementado a mano (<code>procesarDirectorio</code> recursivo)	Delegado al framework (<code>recursive=true</code>)
Deduplicación de archivos	<code>Set<String></code> propio en memoria	Repositorio idempotente propio de Camel (<code>idempotent=true</code>)
Ciclo de polling	<code>while(true) + Thread.sleep</code>	Manejado por el <i>consumer</i> del endpoint (<code>delay=...</code>)
Resiliencia ante errores de red	Baja: un error detiene el proceso completo	Media/alta: un fallo puntual no detiene la ruta
Control fino del protocolo FTP	Alto (acceso directo a <code>FTPClient</code>)	Bajo (se interactúa vía parámetros de URI)
Extensibilidad (agregar pasos al flujo, ej. transformar o enviar a otro destino)	Requiere escribir más código imperativo	Trivial: agregar más <code>.to(...)</code> o componentes de Camel
Curva de aprendizaje	Baja si ya se conoce el protocolo FTP	Media: requiere entender la sintaxis de URIs y EIPs de Camel
Dependencias añadidas al proyecto	<code>commons-net</code> (liviana)	<code>camel-core</code> , <code>camel-file</code> , <code>camel-ftp</code> (más pesado)
Ideal para...	Entender el protocolo a bajo nivel / lógica muy específica no cubierta por el framework	Integraciones productivas que necesiten robustez y sean fáciles de extender

8. Dificultades y soluciones

#	Dificultad encontrada	Solución aplicada
1	<code>FTPClient</code> de Commons Net no ofrece un listado recursivo de directorios de forma nativa.	Se implementó recursión manual (<code>procesarDirectorio</code> se llama a sí misma al encontrar un subdirectorio), filtrando las entradas especiales <code>.</code> y <code>...</code>
2	Evitar volver a descargar un archivo ya procesado en cada ciclo de polling.	Parte A: <code>Set<String></code> en memoria indexado por ruta remota. Parte B: activar <code>idempotent=true</code> en el endpoint FTP de Camel.
3	Por defecto, <code>camel-ftp</code> mueve o elimina el archivo remoto luego de “consumirlo”, lo cual no era deseable para un escenario de monitoreo/réplica (el archivo debía permanecer en el FTP).	Se agregó <code>noop=true</code> al endpoint, indicando a Camel que no debe alterar el archivo origen tras descargarlo.
4	Al construir la URI del endpoint FTP de Camel, usuario y contraseña con caracteres especiales podían romper el parseo de la URI.	Se codificaron con <code>URLEncoder.encode(usuario/password, UTF_8)</code> antes de concatenarlos en la URI.

#	Dificultad encontrada	Solución aplicada
5	Al construir el endpoint <code>file:</code> de destino en Camel, las rutas generadas por <code>Path</code> en Windows usan <code>\</code> y rompían la sintaxis del URI de Camel.	Se normalizó la ruta reemplazando <code>\</code> por <code>/</code> antes de construir el endpoint.
6	Mantener el proceso vivo indefinidamente mientras el polling corre en segundo plano.	Parte A: bucle explícito <code>while(true)</code> en el hilo principal. Parte B: como el <code>CamelContext</code> corre en hilos propios, se bloqueó el hilo principal con <code>new CountDownLatch(1).await()</code> .
7	Cierre abrupto de la ruta de Camel al terminar el proceso (<code>Ctrl+C</code>), dejando hilos o conexiones sin liberar.	Se registró un <code>Runtime.getRuntime().addShutdownHook(...)</code> que llama a <code>contexto.stop()</code> para un apagado ordenado.
8	Necesidad de un servidor FTP de pruebas reproducible sin depender de infraestructura externa ni de instalaciones distintas en cada máquina del equipo.	Se contenerizó un servidor <code>pure-ftpd</code> con Docker Compose, con usuario, puerto y volumen de datos predefinidos en <code>ServidorFTP/docker-compose.yml</code> .
9	La Parte A no tiene manejo de reconexión: un corte de red durante un ciclo de polling termina todo el proceso.	Se documentó como limitación conocida (ver Sección 6) en vez de introducir manejo de errores adicional, para mantener el contraste didáctico frente a la resiliencia que ofrece Camel en la Parte B “out of the box”.

9. Conclusiones

- Ambas implementaciones resuelven correctamente el mismo problema de monitoreo y réplica de archivos desde un servidor FTP, pero desde paradigmas opuestos: **imperativo** (Parte A) vs. **declarativo/EIP** (Parte B).
- **Apache Commons Net (Parte A)** ofrece control total y transparencia sobre cada paso del protocolo FTP, a costa de tener que implementar manualmente aspectos como la recursión, la deduplicación y la resiliencia ante errores.
- **Apache Camel (Parte B)** resuelve el mismo problema con menos código de aplicación, delegando en el framework la recursión, la deduplicación (idempotencia) y una mayor tolerancia a fallos, a cambio de una curva de aprendizaje adicional (sintaxis de URIs y EIPs) y una dependencia más pesada.
- Para un escenario **educativo**, orientado a comprender el protocolo FTP y el ciclo de polling desde cero, la Parte A aporta más valor de aprendizaje. Para un escenario **productivo**, donde se necesite robustez, extensibilidad y menor tiempo de desarrollo, la Parte B (Camel) es la opción más adecuada.
- La externalización de la configuración y la contenerización del servidor FTP con Docker fueron decisiones clave que permitieron comparar ambas soluciones bajo las mismas condiciones, de forma reproducible.

10. Pruebas

